Report 2: Reducing Runtime in Floquet NESS Engineering

From a T-grid outer loop to fast joint optimization in (θ, T)

Abdrakhman Akchurin

February 2026

Context. This report is a follow-up to the February 2, 2026 write-up "Introduction to Floquet NESS Engineering with Quantum Optimal Control" (Report 1). In particular, Report 1 introduced the Floquet map construction, augmented fixed-point solve, and the T-scan strategy for selecting T.

1 Recap: what stays the same (physics and objective)

We study an open, periodically-driven qubit governed by a Lindblad master equation

$$\frac{\mathrm{d}\rho(t)}{\mathrm{d}t} = \mathcal{L}(t)\rho(t), \tag{1}$$

with period T, i.e. $\mathcal{L}(t+T) = \mathcal{L}(t)$. Define the one-period propagator (superoperator) $\Phi(\theta, T)$ such that

$$\rho(T) = \Phi(\theta, T)\rho(0). \tag{2}$$

A Floquet NESS is a fixed point

$$\rho_* = \Phi(\theta, T)\rho_*, \qquad \mathrm{Tr}(\rho_*) = 1. \tag{3}$$

In practice we compute ρ_* via the augmented linear system described in Report 1 ("fixed point solve").

Control choice in this report. To focus on runtime and algorithmics, the experiments in this report use a *Jx-only* control:

$$H(t) = \frac{1}{2} J_x(t)\sigma_x, \tag{4}$$

with dissipation fixed (relaxation + dephasing, as in Report 1). The optimization target is the stroboscopic NESS polarization

$$J(\theta, T) \equiv \langle \sigma_z \rangle_{\rho_*} = \mathrm{Tr}(\sigma_z \rho_*), \tag{5}$$

and we *minimize* J.

2 Problem with the previous procedure: why runtime blew up

Report 1 proposed selecting the period T using an outer-loop scan over a discrete grid T_{grid} and, at each grid point, running an inner optimization over waveform parameters θ. That decision was motivated by robustness and clean presentation, but it has an important practical drawback: it multiplies the total compute time by the number of T values in the grid.

2.1 Bottleneck 1: the T-grid multiplies the total work

Let $K = |T_{\text{grid}}|$ be the number of candidate periods, and let each inner optimization take I iterations (and possibly S random restarts). Then, even if the cost per iteration is modest,

$$\text{Total cost of } T\text{-scan } \sim\ K \times S \times I \times \big(\text{cost per iteration}\big). \tag{6}$$

In other words, *a fine grid can dominate runtime*, especially if the objective landscape in T is broad and you need large K to resolve it.

2.2 Bottleneck 2: "clean" gradients can require one solve per parameter

Report 1 derived the gradient of the fixed point via implicit differentiation, leading to an augmented linear system that must be solved once for each parameter θ_k. This is mathematically clean, but algorithmically expensive when the number of parameters grows (e.g., multiple controls, larger Fourier truncation, or time-sliced controls).

Even in the single-qubit case (small matrices), the repeated solves and repeated derivative-through-exponential calls can create large Python-level overhead.

2.3 Practical consequence

The runtime was dominated by *repetition*:

- repetition across T values (outer loop), and
- repetition across parameters (gradient evaluation).

The new approach below removes the first repetition entirely and compresses the second repetition to "one expensive operation per time slice" rather than "one per time slice per parameter."

3 New approach: fast joint optimization in (θ, T)

The new procedure treats T as an optimization variable and updates it jointly with θ:

$$(\theta^*, T^*) \in \arg \min_{\theta \in \Theta,\ T \in [T_{\min}, T_{\max}]} J(\theta, T). \tag{7}$$

This removes the outer-loop T-scan. The key technical requirement is that we must provide the optimizer with a gradient in *both* variables:

$$\nabla_\theta J(\theta, T), \qquad \frac{\partial J}{\partial T}(\theta, T).$$

3.1 Parameterization used in code: $N = 48$, $M = 3$, Fourier in normalized time

We work in normalized time $s = t/T \in [0, 1)$ and discretize one period into N slices at midpoints

$$s_n = \frac{n - \frac{1}{2}}{N}, \qquad n = 1, \ldots, N, \qquad \Delta t = \frac{T}{N}. \tag{8}$$

The Jx-only waveform is represented by a Fourier series truncated at $M = 3$:

$$J_x(s) = a_0 + \sum_{m=1}^{M} \Big(a_m \cos(2\pi m s) + b_m \sin(2\pi m s) \Big), \qquad M = 3. \tag{9}$$

We denote $\theta = (a_0, a_1, b_1, a_2, b_2, a_3, b_3) \in \mathbb{R}^{2M+1}$.

Why this helps. Because the basis functions depend only on s, the mapping from Fourier coefficients θ to slice values $J_{x,n} = J_x(s_n)$ is independent of T. This is one reason joint optimization remains numerically stable.

4 How the fast objective/gradient evaluation works now

This section is the main "what changed" explanation. One evaluation of the objective and gradient at a candidate (θ, T) contains a forward pass plus an efficient backward/adjoint pass.

4.1 Forward pass: build Φ and solve for the Floquet NESS

At given (θ, T):

1. **Compute slice amplitudes.** Evaluate $J_{x,n} = J_x(s_n)$ on all N slices.
2. **Build slice Liouvillians cheaply.** Use an affine form
$$L_n = L_{\text{diss}} + J_{x,n} L_x, \tag{10}$$
where L_{diss} and L_x are precomputed once.
3. **Compute step maps and the one-period map.**
$$U_n = \exp(\Delta t\, L_n), \qquad \Phi = U_N U_{N-1} \cdots U_1. \tag{11}$$
4. **Solve for the fixed point.** Obtain $r_* = \text{vec}(\rho_*)$ from the augmented system described in Report 1 (fixed point + trace constraint).
5. **Compute the objective.** Return $J = \text{Tr}(\sigma_z \rho_*)$.

What forms of the Liouvillian appear in our problem? In general, the Lindblad master equation can be written as

$$\dot{\rho}(t) = \mathcal{L}(t)[\rho(t)] = -i[H(t), \rho(t)] + \sum_k \Big(L_k \rho(t) L_k^\dagger - \tfrac{1}{2}\{L_k^\dagger L_k, \rho(t)\} \Big), \tag{12}$$

so the Liouvillian $\mathcal{L}(t)$ is the sum of a **Hamiltonian (coherent)** part and a **dissipative (Lindblad)** part. In our implementation we work in Liouville space by vectorizing the density matrix $r = \text{vec}(\rho)$, so the equation becomes a linear ODE

$$\dot{r}(t) = L(t)\, r(t), \tag{13}$$

where $L(t)$ is a $D \times D$ matrix ($D = d^2 = 4$ for a qubit).

(1) Hamiltonian / commutator superoperator. The coherent contribution has the form

$$\mathcal{L}_H(t)[\rho] = -i[H(t), \rho]. \tag{14}$$

Under column-stacking vectorization, this becomes

$$L_H(t) = -i\big(\mathbb{I} \otimes H(t) - H(t)^{\mathsf{T}} \otimes \mathbb{I}\big). \tag{15}$$

In this report we use a *single control channel* with

$$H(t) = \tfrac{1}{2} J_x(t) \sigma_x, \tag{16}$$

so the Hamiltonian contribution is linear in the scalar amplitude $J_x(t)$:

$$L_H(t) = J_x(t)\, L_x, \qquad L_x \equiv -\frac{i}{2}\big(\mathbb{I} \otimes \sigma_x - \sigma_x^{\mathsf{T}} \otimes \mathbb{I}\big). \tag{17}$$

This defines the fixed matrix L_x that we precompute once.

(2) Dissipative Lindblad superoperator. The dissipative contribution is the sum of Lindblad dissipators

$$\mathcal{L}_{\text{diss}}[\rho] = \sum_k \mathcal{D}[L_k]\rho, \qquad \mathcal{D}[L]\rho \equiv L\rho L^\dagger - \tfrac{1}{2}\{L^\dagger L, \rho\}. \tag{18}$$

In our qubit experiments we include (i) amplitude damping with $L_\downarrow = \sqrt{\gamma_\downarrow}\,\sigma_-$ and (ii) pure dephasing with $L_\phi = \sqrt{\gamma_\phi}\,\sigma_z$:

$$\mathcal{L}_{\text{diss}}[\rho] = \gamma_\downarrow\, \mathcal{D}[\sigma_-]\rho + \gamma_\phi\, \mathcal{D}[\sigma_z]\rho. \tag{19}$$

After vectorization this becomes a constant 4×4 matrix L_{diss} that we also precompute once.

(3) The affine (control-separable) Liouvillian used in code. Putting the two parts together, the full Liouvillian matrix is

$$L(t) = L_{\text{diss}} + J_x(t)\, L_x. \tag{20}$$

After time discretization into slices, this becomes

$$L_n = L_{\text{diss}} + J_{x,n}\, L_x. \tag{21}$$

This *affine* form (constant term + scalar times a fixed matrix) is what makes the implementation fast: once L_{diss} and L_x are built, each slice requires only a scaled matrix addition rather than reconstructing Lindblad/Kronecker structure from scratch.

4.2 Speedup idea 1: compute gradients w.r.t. slice amplitudes first

A direct approach would differentiate w.r.t. each Fourier coefficient (a_m, b_m). Instead, we compute a gradient w.r.t. the slice amplitudes $J_{x,n}$:

$$g_n \equiv \frac{\partial J}{\partial J_{x,n}}, \qquad n = 1, \ldots, N, \tag{22}$$

then convert that to $\nabla_\theta J$.

Fourier projection Let the Fourier-coefficient vector be

$$\theta \equiv \begin{bmatrix} a_0 & a_1 & b_1 & a_2 & b_2 & \cdots & a_M & b_M \end{bmatrix}^{\mathsf{T}} \in \mathbb{R}^p, \qquad p = 2M + 1,$$

and define the vector of slice amplitudes

$$J_{x,\cdot} \equiv \begin{bmatrix} J_{x,1} & J_{x,2} & \cdots & J_{x,N} \end{bmatrix}^{\mathsf{T}} \in \mathbb{R}^N, \qquad J_{x,n} = J_x(s_n).$$

We introduce the *Fourier design matrix* $B \in \mathbb{R}^{N\times p}$ whose n-th row contains the Fourier basis functions evaluated at s_n:

$$B \equiv \begin{bmatrix} 1 & \cos(2\pi s_1) & \sin(2\pi s_1) & \cos(4\pi s_1) & \sin(4\pi s_1) & \cdots & \cos(2\pi M s_1) & \sin(2\pi M s_1) \\ 1 & \cos(2\pi s_2) & \sin(2\pi s_2) & \cos(4\pi s_2) & \sin(4\pi s_2) & \cdots & \cos(2\pi M s_2) & \sin(2\pi M s_2) \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 1 & \cos(2\pi s_N) & \sin(2\pi s_N) & \cos(4\pi s_N) & \sin(4\pi s_N) & \cdots & \cos(2\pi M s_N) & \sin(2\pi M s_N) \end{bmatrix},$$

i.e.

$$B_{n,0} = 1, \qquad B_{n,2m-1} = \cos(2\pi m s_n), \qquad B_{n,2m} = \sin(2\pi m s_n), \quad m = 1, \ldots, M.$$

With this definition, evaluating the Fourier series on the slice grid is the linear map

$$J_{x,\cdot} = B\,\theta. \tag{23}$$

Now define the slice-gradient vector

$$g \equiv \begin{bmatrix} \frac{\partial J}{\partial J_{x,1}} & \frac{\partial J}{\partial J_{x,2}} & \cdots & \frac{\partial J}{\partial J_{x,N}} \end{bmatrix}^{\mathsf{T}} \in \mathbb{R}^N.$$

Chain rule in matrix (differential) form. Because the objective depends on θ only through the slice amplitudes $J_{x,\cdot} = B\theta$, a small change $d\theta$ produces a change in the slice vector

$$dJ_{x,\cdot} = B\,d\theta. \tag{24}$$

By definition of the slice-gradient g, the corresponding change in the scalar objective is

$$dJ = g^{\mathsf{T}}\,dJ_{x,\cdot}. \tag{25}$$

Substituting $dJ_{x,\cdot} = B\,d\theta$ gives

$$dJ = g^{\mathsf{T}} B\,d\theta. \tag{26}$$

On the other hand, the definition of the gradient with respect to θ is

$$dJ = (\nabla_\theta J)^{\mathsf{T}} d\theta. \tag{27}$$

Since these two expressions hold for all perturbations $d\theta$, we conclude

$$(\nabla_\theta J)^{\mathsf{T}} = g^{\mathsf{T}} B \qquad \Longrightarrow \qquad \nabla_\theta J = B^{\mathsf{T}} g. \tag{28}$$

This is exactly the matrix form of the component-wise chain rule

$$\frac{\partial J}{\partial \theta_k} = \sum_{n=1}^{N} \frac{\partial J}{\partial J_{x,n}} \frac{\partial J_{x,n}}{\partial \theta_k}.$$

4.3 Speedup idea 2: prefix/suffix products (cheap product-rule derivatives)

To get g_n we need the sensitivity of Φ to the n-th slice. Define prefix/suffix products

$$P_{n-1} = U_{n-1} \cdots U_1, \qquad S_{n+1} = U_N \cdots U_{n+1}.$$

Then the product rule gives

$$\frac{\partial \Phi}{\partial J_{x,n}} = S_{n+1} \left(\frac{\partial U_n}{\partial J_{x,n}} \right) P_{n-1}. \tag{29}$$

We compute all P_n and S_n once per iteration; assembling each $\partial\Phi/\partial J_{x,n}$ is then only a small number of matrix multiplications.

4.4 Speedup idea 3: obtain $\partial U_n/\partial J_{x,n}$ from one block exponential

Each step map is $U_n = \exp(A_n)$ with $A_n = \Delta t\, L_n$. The directional derivative we need is

$$\frac{\partial U_n}{\partial J_{x,n}} = D\exp(A_n)[E], \qquad E = \Delta t\, L_x.$$

Rather than calling separate "Fréchet derivative" routines, we compute U_n and its derivative together via the block-matrix identity

$$\exp\begin{pmatrix} A_n & E \\ 0 & A_n \end{pmatrix} = \begin{pmatrix} \exp(A_n) & D\exp(A_n)[E] \\ 0 & \exp(A_n) \end{pmatrix}. \tag{30}$$

For a single qubit, A_n acts on a 4D Liouville space, so the block exponential is only 8×8, which is fast in practice.

4.5 Speedup idea 4: one adjoint solve instead of one solve per parameter (with derivation)

The remaining issue is: even if we can compute $\partial\Phi/\partial J_{x,n}$, how do we obtain the scalar slice sensitivity $g_n = \partial J/\partial J_{x,n}$ without solving a new linear system for every slice (or every Fourier parameter)?

Augmented fixed-point system (derivation). The Floquet NESS $r_* = \mathrm{vec}(\rho_*)$ satisfies the fixed-point equation

$$r_* = \Phi r_* \quad \Longleftrightarrow \quad (I - \Phi) r_* = 0.$$

Because a fixed point exists, $(I - \Phi)$ is singular. To select the unique *normalized* fixed point we also impose the trace constraint $\mathrm{Tr}(\rho_*) = 1$. Under column-stacking vectorization, $\mathrm{Tr}(\rho) = c^{\mathsf{T}} r$ with $c \equiv \mathrm{vec}(I)$, hence $c^{\mathsf{T}} r_* = 1$. We combine these conditions into a single square linear system by introducing an auxiliary scalar λ:

$$(I - \Phi) r_* + c\,\lambda = 0, \qquad c^{\mathsf{T}} r_* = 1, \tag{31}$$

which can be written as the augmented system

$$A(\Phi)\, x = b, \qquad A(\Phi) \equiv \begin{pmatrix} I - \Phi & c \\ c^{\mathsf{T}} & 0 \end{pmatrix}, \quad x \equiv \begin{pmatrix} r_* \\ \lambda \end{pmatrix}, \quad b \equiv \begin{pmatrix} 0 \\ 1 \end{pmatrix}. \tag{32}$$

We solve this once per objective/gradient evaluation to obtain r_*.

Adjoint derivation of g_n. Our objective is a linear functional of the fixed point,

$$J = \langle \sigma_z \rangle_{\rho_*} = \mathrm{Tr}(\sigma_z \rho_*) = \Re\Big[O^\dagger r_*\Big], \qquad O \equiv \mathrm{vec}(\sigma_z),$$

and we extend O to the augmented variable by defining $y \equiv \left(\begin{smallmatrix} O \\ 0 \end{smallmatrix}\right)$ so that $J = \Re[y^\dagger x]$.

Let p denote any parameter (here $p = J_{x,n}$). Differentiating $A(\Phi(p))x(p) = b$ gives

$$A\,\frac{\partial x}{\partial p} + \frac{\partial A}{\partial p} x = 0 \quad \Longrightarrow \quad \frac{\partial x}{\partial p} = -A^{-1}\left(\frac{\partial A}{\partial p}\right) x.$$

Therefore,

$$\frac{\partial J}{\partial p} = \Re\left[y^\dagger \frac{\partial x}{\partial p}\right] = -\Re\left[y^\dagger A^{-1}\left(\frac{\partial A}{\partial p}\right) x\right].$$

To avoid applying A^{-1} separately for each parameter, we introduce the adjoint vector w defined by one adjoint solve

$$A^\dagger w = y, \tag{33}$$

so that $y^\dagger A^{-1} = w^\dagger$. This yields the adjoint formula

$$\frac{\partial J}{\partial p} = -\Re\left[w^\dagger \left(\frac{\partial A}{\partial p}\right) x\right].$$

In our case, A depends on p only through Φ in the $(1,1)$ block, so

$$\frac{\partial A}{\partial p} = \begin{pmatrix} -\frac{\partial \Phi}{\partial p} & 0 \\ 0 & 0 \end{pmatrix}.$$

Substituting this into the previous expression cancels the minus sign and eliminates the λ-block, giving

$$\frac{\partial J}{\partial p} = \Re\left[w_{\text{top}}^\dagger \left(\frac{\partial \Phi}{\partial p}\right) r_*\right], \tag{34}$$

where w_{top} denotes the first $D{=}4$ components of w (corresponding to r_*). Finally, taking $p = J_{x,n}$ defines the slice gradient used in the algorithm:

$$g_n \equiv \frac{\partial J}{\partial J_{x,n}} = \Re\left[w_{\text{top}}^\dagger \left(\frac{\partial \Phi}{\partial J_{x,n}}\right) r_*\right]. \tag{35}$$

Computational payoff. We factorize $A(\Phi)$ once and reuse it to solve both the forward system (for r_*) and the adjoint system (for w). After that, each g_n is obtained by a matrix multiplication/contraction, so we replace "one augmented solve per parameter" with *one* extra solve per iteration.

4.6 Optimizing the period T: Option A complex-step derivative

We update T jointly with θ, but we avoid a complicated analytic expression for $\partial J/\partial T$ by using a complex-step derivative:

$$\frac{\partial J}{\partial T}(T) \approx \frac{\Im\big(\widetilde{J}(T + \mathrm{i}h)\big)}{h}, \qquad h \ll 1, \tag{36}$$

where $\widetilde{J}$ denotes the same objective pipeline evaluated in complex arithmetic. This behaves like "one extra objective evaluation per iteration" and avoids finite-difference subtraction error.

5 Optimizer workflow (what happens each iteration)

We use a bound-constrained quasi-Newton method (L-BFGS-B) on the concatenated variable

$$x = (\theta, T) \in \mathbb{R}^{(2M+1)+1}.$$

Each optimizer iteration performs the following steps:

Algorithm (one iteration). Given current (θ, T):

1. Forward pass: evaluate slice amplitudes $\{J_{x,n}\}$, build $\{U_n\}$ and Φ, solve for r_*, compute $J = \mathrm{Tr}(\sigma_z \rho_*)$.
2. Adjoint pass: reuse the augmented-system factorization to obtain the adjoint vector w.
3. Slice gradients: for $n = 1, \ldots, N$, compute $\partial U_n / \partial J_{x,n}$ (block exponential), assemble $\partial \Phi / \partial J_{x,n}$ using prefix/suffix products, and contract to obtain g_n.
4. Fourier gradient: compute $\nabla_\theta J = B^\top g$.
5. Period gradient: compute $\partial J / \partial T$ via complex-step.
6. Return $(J, \nabla_\theta J, \partial J / \partial T)$ to L-BFGS-B and update (θ, T).

Multi-start (to reduce local-minimum sensitivity). Because the landscape can be nonconvex, we run multiple random restarts within parameter bounds and keep the best solution.

6 Concrete cost comparison (old vs new) for our experiment

This section explains the improvement at the level of "how many expensive operations" are performed.

6.1 Experiment settings

We use $N = 48$ slices and $M = 3$ Fourier truncation, so the number of Fourier parameters is $p = 2M + 1 = 7$.

6.2 Old pattern (Report 1 style)

- A T-grid with K values multiplies total work by K (and by restarts).
- Clean implicit differentiation can require one augmented solve per parameter (up to p solves).
- Derivative-through-exponentials can scale like $N \times p$ directional derivative evaluations.

6.3 New pattern (this report)

- No T-grid: (θ, T) are optimized jointly in one loop.
- One LU factorization of the augmented system reused for two solves (forward + adjoint).
- "One expensive derivative per slice" (N block exponentials) to obtain the entire slice-gradient g, followed by a cheap projection $B^\top g$.
- Period optimization via complex-step adds roughly one extra objective evaluation per iteration.

Bottom line. The new method removes the multiplicative K factor from a T-scan and replaces parameter-wise linear solves with a single adjoint solve per iteration. The resulting runtime is dominated by $O(N)$ small matrix exponentials per iteration, which is tractable for the single-qubit case and scales more favorably as control dimension grows.

7 Post-optimization diagnostics (what we plot and why)

After obtaining (θ^*, T^*), we generate diagnostics to validate the optimized solution:

7.1 Optimized $J_x(t)$ over four periods

We evaluate the optimized waveform over $t \in [0, 4T^*]$ using $s = (t/T^*) \bmod 1$ and the Fourier series. This confirms periodicity and shows the smoothness implied by $M = 3$.

7.2 Micromotion from the Floquet NESS over four periods

The NESS is defined stroboscopically ($t = kT^*$), but there can be intra-period motion. Starting from $r_0 = \mathrm{vec}(\rho_*)$, propagate through the step maps for $4N$ steps:

$$r_{k+1} = U_{(k \bmod N)+1}\, r_k,$$

and compute the Bloch components $\langle\sigma_x\rangle(t)$, $\langle\sigma_y\rangle(t)$, $\langle\sigma_z\rangle(t)$ at each step. Plot each component separately over four periods.

7.3 Purity over four periods

Along the same micromotion trajectory, compute and plot

$$\mathcal{P}(t) = \mathrm{Tr}(\rho(t)^2).$$

This quantifies how mixedness varies within each cycle even when the stroboscopic state is stationary.

7.4 Stroboscopic convergence

To confirm that ρ_* is an attractive fixed point, initialize a random physical density matrix ρ_0 and iterate the one-period map:

$$r_{k+1} = \Phi r_k.$$

Plot the Frobenius distance $\|\rho_k - \rho_*\|_F$ and the observable sequence $\langle\sigma_z\rangle_k$ versus k.

8 Notes and next steps

- **Convergence in N.** Although $N = 48$ is sufficient for $M = 3$ in our runs, results should be verified by increasing N and checking stability of (θ^*, T^*, J^*).
- **Extending controls.** The same runtime ideas (slice-gradient + adjoint) generalize cleanly if we include $J_y(t)$ and/or $\Delta(t)$.
- **Optional analytic dJ/dT.** The complex-step derivative is robust and easy; if needed, it can be replaced by an analytic dJ/dT computed with the same adjoint framework (at additional per-iteration cost).